WEEK 2 · LESSON 7 OF 24 · HANDOUT

CI/CD for Machine Learning

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=HMlXSiDKWrg — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2020: Can we deploy yet? — https://www.youtube.com/watch?v=Fgyk8P0IJsU

🎓 **Go deeper:** PyCascades 2024: Embracing MLOps — Bridging the Gap Between Machine Learning and DevOps — https://www.youtube.com/watch?v=wRnKcWIZEvk

Your goal in this lesson: Explain continuous integration and continuous delivery for ML, write a quality-gate script that blocks bad models, and read a GitHub Actions workflow that runs it automatically.

From "I ran it on my laptop" to automatic checks

Continuous integration (CI) means: every time someone pushes code, a server automatically installs everything and runs the tests. **Continuous delivery (CD)** means: when the tests pass, the new version is packaged (and possibly deployed) automatically. For ML we add one more step — a **quality gate** that checks the model itself, not just the code.

Step	What runs	Stops the release if…
Lint and unit tests	Code checks, feature tests	The code is broken
Data validation	Schema and range checks	The training data is wrong
Train	The training script with the config	Training crashes
Quality gate	Metrics and behaviour tests	The model is worse than required or than production
Package	Docker build with a version tag	The image does not build
Deploy	To staging first, then production	Smoke tests on staging fail

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

A quality-gate script

A gate is a small program that **exits with code 0 when everything is fine and 1 when not**. CI systems read that exit code.

```
import joblib, json, subprocess, sys

joblib.dump(model, "candidate.joblib")
test.to_csv("holdout.csv", index=False)
json.dump({"auc": 0.87}, open("production_metrics.json", "w"))      # what the live model
achieves

gate = '''
import json, sys
import joblib, pandas as pd
from sklearn.metrics import roc_auc_score

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
MIN_AUC = 0.80          # absolute minimum
MAX_DROP = 0.01         # may not be more than 0.01 worse than production

model = joblib.load(sys.argv[1])
data = pd.read_csv("holdout.csv")
production = json.load(open("production_metrics.json"))
p = model.predict_proba(data[FEATURES])[:, 1]
auc = roc_auc_score(data["repaid"], p)

base = data[FEATURES].iloc[[0]].copy()
worse = base.assign(late_payments=base["late_payments"] + 3)
checks = {
    f"AUC {auc:.3f} >= {MIN_AUC}": auc >= MIN_AUC,
    f"AUC not more than {MAX_DROP} below production ({production['auc']})": auc >=
production["auc"] - MAX_DROP,
    "more late payments lowers the probability": model.predict_proba(worse)[0, 1] <
model.predict_proba(base)[0, 1],
}
for name, ok in checks.items():
    print("PASS" if ok else "FAIL", name)
sys.exit(0 if all(checks.values()) else 1)
'''
open("quality_gate.py", "w").write(gate)

result = subprocess.run([sys.executable, "quality_gate.py", "candidate.joblib"],
capture_output=True, text=True)
print(result.stdout, "exit code:", result.returncode)
```

The gate blocks a weaker model

A colleague "simplifies" the model by dropping three features:

```
from sklearn.compose import ColumnTransformer

few = ["income_rwf", "loan_rwf", "salaried"]
small = make_pipeline(ColumnTransformer([("keep", "passthrough", few)]),   # uses only 3
columns
                      StandardScaler(), LogisticRegression(max_iter=1000))
small.fit(train[FEATURES], train[TARGET])
joblib.dump(small, "candidate_small.joblib")

result = subprocess.run([sys.executable, "quality_gate.py", "candidate_small.joblib"],
capture_output=True, text=True)
print(result.stdout, "exit code:", result.returncode)
```

The smaller model's AUC (about 0.87) is still good enough — but it **ignores late payments completely**, so the behaviour check fails and the gate returns exit code **1**. CI stops the release automatically. A score-only gate would have let this model through; that is why gates check behaviour too.

A GitHub Actions workflow

Save this as `.github/workflows/ml.yml` in your repository. GitHub runs it on every push:

```
name: train-and-check
on: [push, pull_request]
jobs:
  ml:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python -m pytest -q tests/            # code, data and API tests
      - run: python src/train.py --config configs/train.json --out candidate.joblib
      - run: python quality_gate.py candidate.joblib
      - run: docker build -t loan-service:${{ github.sha }} .
```

If any step fails, the workflow shows a red cross and the later steps (like building the image) do not run.

Good CI/CD habits

- Keep the pipeline **fast**: train on a sample in CI, the full data on a schedule.
- Deploy to **staging** first and run smoke tests (call `/health` and one `/predict`).
- Keep the previous image so you can **roll back** in one command.
- Store gate thresholds in the repository, so changing them is visible and reviewed.

Key terms

- **Continuous integration (CI)** — automatic tests on every change.
- **Continuous delivery (CD)** — automatic packaging and release after tests pass.
- **Quality gate** — a script that blocks models that are not good enough.
- **Exit code** — 0 means success, any other number means failure.
- **Staging** — a copy of production for final checks.
- **Rollback** — going back to the previous version.

Remember this: CI/CD for ML runs tests, trains, and passes every model through an automatic quality gate before packaging — bad models are stopped by machines, not memory.

Name: _____ Date: _____

Activity

Task 1

Add a check to the gate that the share of approved applicants is between 40% and 90%.

Task 2

Set `MAX_DROP` to 0.05. Does the small model pass now? Is that a good idea?

Task 3

Write the smoke test that staging should pass, as a short Python function using TestClient.

Task 4

Create a free GitHub repository with the workflow above and watch it run.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is continuous integration (CI)?

   A. Writing documentation
   B. Training once a year
   C. Automatically running tests on every code change
   D. Buying servers

2. What does a quality gate do?

   A. Blocks models that are not good enough from being released
   B. Speeds up training
   C. Draws charts
   D. Encrypts data

3. What exit code means success?

   A. 200
   B. 422
   C. 0
   D. 1

4. Why did the smaller model fail the gate, even with a good AUC?

   A. It used too many features
   B. It was too big
   C. It ignores late payments, so the behaviour check failed
   D. It had no name

5. Where should a new version go before production?

   A. Nowhere
   B. Directly to all users
   C. A USB stick
   D. Staging, for smoke tests

6. Why keep gate thresholds in the repository?

   A. To make CI slower
   B. Git needs them
   C. They are secret
   D. Changes to them become visible and reviewed

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.